What's Next?

You now know how to build dynamic, server-powered bridge components for iOS. You know how to keep configuration in HTML, how to send messages, and how to reply back. You can use the same techniques to build your own components, taking advantage of any native API.

These bridge components provide a great compromise between web-only screens and native ones. They enable native interactions without fully investing in screens completely driven by SwiftUI. When built correctly, they can provide endless customization without new releases to the app stores—all that's required is changing some HTML.

If you're looking for more examples or inspiration, check out my open source bridge component library.[47] It includes a bunch of generalized, production-ready bridge components extracted from real-world client projects. Once configured, each component can be added to any page of your app and customized with a bit of HTML.

Up next, we'll build the same bridge component on Android. If you're only working on iOS right now, feel free to skip ahead to Chapter 9, *Deploy to Physical Devices with TestFlight and Play Testing*, to learn how to get the app off of the simulator and into your hands.

Footnotes

[37] https://github.com/hotwired/hotwire-native-ios/tree/main/Demo

[38] https://github.com/hotwired/hotwire-native-ios/tree/main/Demo

[39] https://github.com/hotwired/hotwire-native-android/tree/main/demo

[40] https://stimulus.hotwired.dev

[41] https://stimulus.hotwired.dev/handbook/introduction

[42]